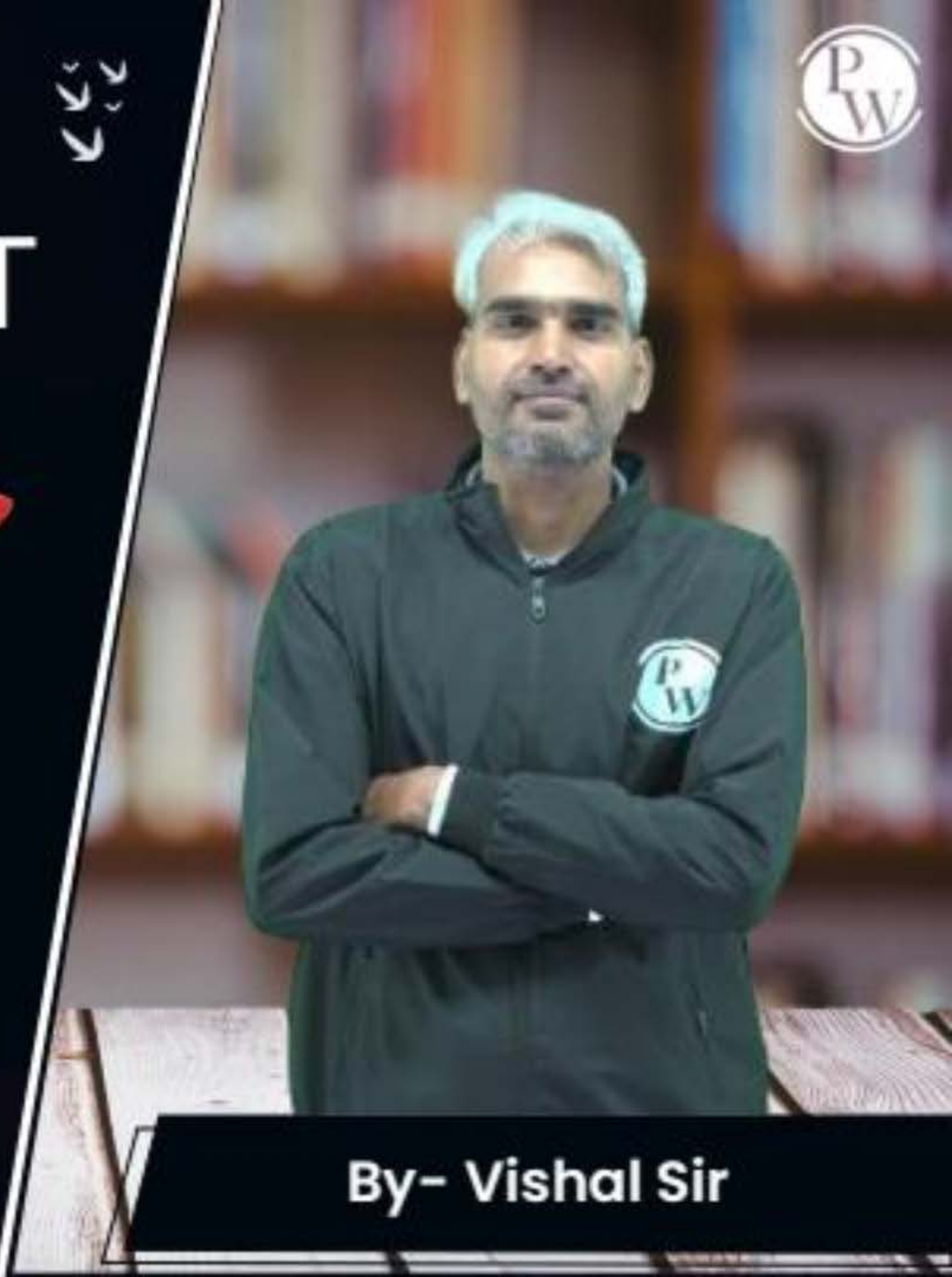
Computer Science & IT



Database Management System

File organization & Indexing

Lecture No. 06



By- Vishal Sir

Recap of Previous Lecture



Topic B Tree Index



Slide

Topics to be Covered



Topic	B+ Tree Index

Slide

#Q. Consider a B tree with order = p

a) Find the minimum number of nodes required in B tree of order =p and height = h

b) Find the minimum number of keys required in B tree of order =p and height = h

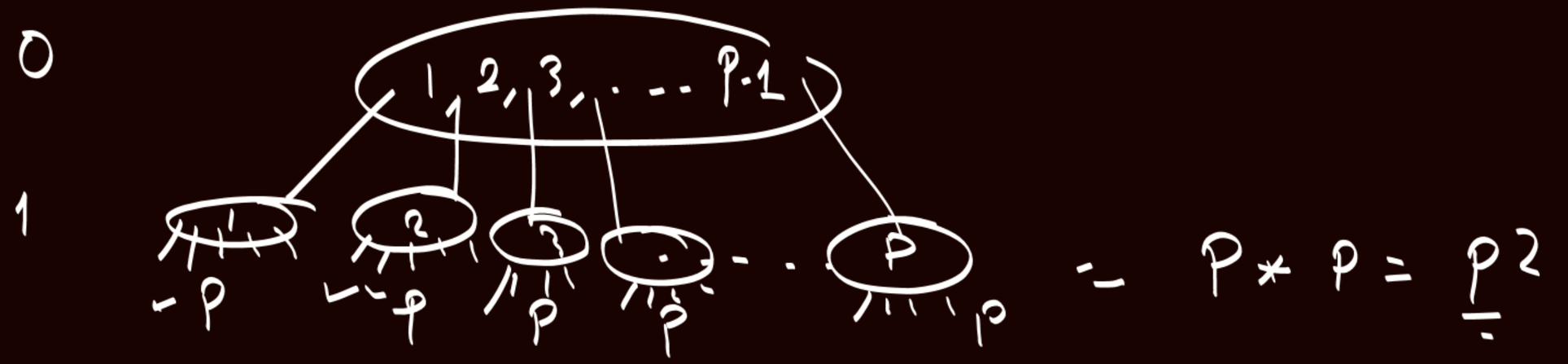
c) Find the maximum number of nodes possible in B tree of order =p and height = h

b) Find the maximum number of keys possible in B tree of order =p and height = h

Height/Level	Min. No. of nodes	Min. no of Pointers	Min no. of Keys	Max. No. of nodes	Max. no. of Pointers	Max. No. of keys
at 0/1	1	2	1	1	p	(p-1)
at 1/2	$2*\lceil \frac{p}{2} \rceil^0$	$2*\lceil \frac{p}{2} \rceil^1$	$2*(\lceil \frac{p}{2} \rceil - 1)$	p	$p*p = p^2$	$p*(p-1)$
at 2/3	$2*\lceil \frac{p}{2} \rceil$	$(2*\lceil \frac{p}{2} \rceil)*\lceil \frac{p}{2} \rceil$	$2*\lceil \frac{p}{2} \rceil*(\lceil \frac{p}{2} \rceil - 1)$	p^2	$p^2*p = p^3$	$p^2*(p-1)$
at 3/4	$2*\lceil \frac{p}{2} \rceil^2$	$(2*\lceil \frac{p}{2} \rceil^2)*\lceil \frac{p}{2} \rceil$	$2*\lceil \frac{p}{2} \rceil^2*(\lceil \frac{p}{2} \rceil - 1)$	p^3	$p^3*p = p^4$	$p^3*(p-1)$
h/(h+1)	$2*\lceil \frac{p}{2} \rceil^{h-1}$	$2*\lceil \frac{p}{2} \rceil^h$	$(2*\lceil \frac{p}{2} \rceil)^{h-1}*(\lceil \frac{p}{2} \rceil - 1)$	$= \sum p^h$	$= \sum p^{h+1}$	$= \sum p^h*(p-1)$

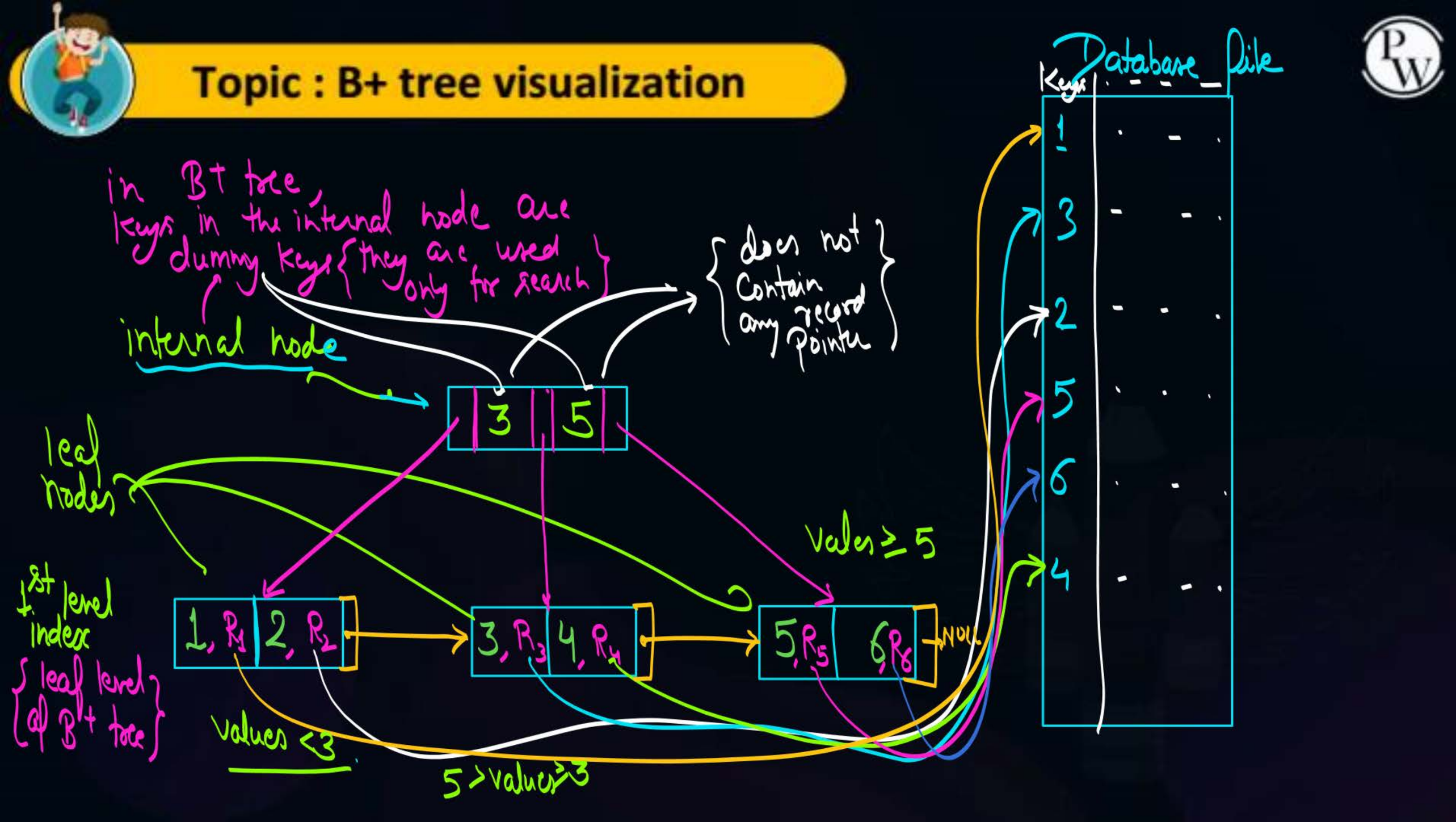
Summation of all terms will give us Min No. of nodes in a Btree of order p & height = h

Au :- $\sum$



$= p * p = p^2$

Topic : B+ tree visualization





Topic : B+ Tree

- B+ tree is some what similar to normal multi-level index.
- In B+ Tree all keys along with their record pointers are present at leaf nodes of B+ Tree.
- In B+ Tree keys in the internal nodes are used only for search operation (they are dummy keys), they are not used to point any record of the database as no record pointer is available in internal nodes of B+ Tree.



Topic : Structure of internal node of B+ Tree

∴ No record pointer in the node {only dummy keys}

Block Pointers Pointing to the next level index blocks



Block with keys $< K_1$

Block with $K_1 \leq$ keys $< K_2$

Block with $K_2 \leq$ keys $< K_3$

Block with $K_{q-2} \leq$ keys $< K_{q-1}$

Block with $K_{q-1} \leq$ keys



Topic : Order of Internal node of B+ Tree

Order of an internal node of B+ tree is defined as the maximum number of child pointer (block pointer/ node pointer) a B+ tree node can have. (same as B Tree)

- Let order of a node of B+ tree is denoted by 'q'
- One internal not of B+ tree must fit in a single disk block

let order of internal node of B+ tree : q

∴

$$(q * \text{Block pointer size}) + ((q-1) * (\text{key size})) \leq \text{Disk block size}$$



Topic : Order of Internal node of B+ Tree

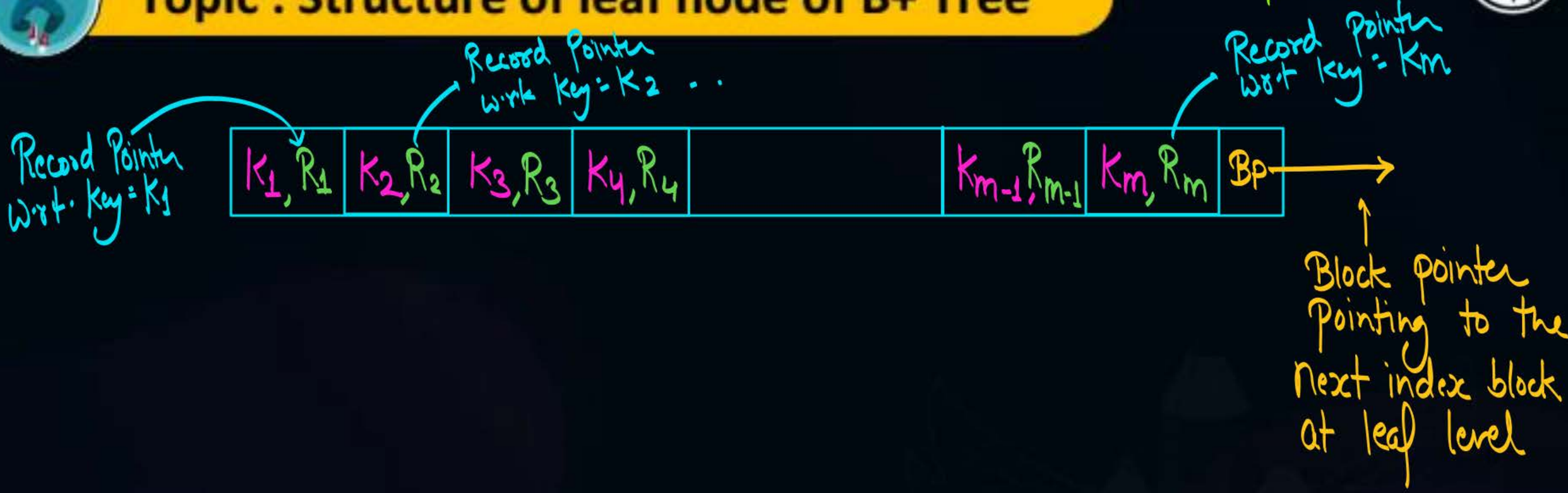
Let 'q' is the order of an internal node of B+ Tree

- Maximum number of child pointer a node can have is = q
- Maximum number of keys a node can have is = q-1
- Minimum number of child pointer a non-root node can have is = $\lceil q/2 \rceil$
- Minimum number of Keys a non-root node can have is = $\lceil q/2 \rceil$-1
- Minimum number of child pointer root node can have is = 2
- Minimum number of keys root node can have is = 1



Topic : Structure of leaf node of B+ Tree

→ will Contain keys along with record Pointers





Topic : Order of leaf node of B+ Tree

Order of leaf a node of B+ tree is defined as the maximum number of keys a B+ tree node can have. (different from B Tree)

leaf

leaf

- Let order of a node of B+ tree is denoted by 'm'
- One leaf node of B+ tree should also fit in a single disk block
- let order of leaf node of B+ tree is 'm'

∴

$$\left(m * (\text{Key Size} + \text{Record Pointer Size})\right) + 1 * (\text{Block Pointer Size}) \leq \text{Disk block Size}$$



Topic : Order of leaf node of B+ Tree

All records with keys b/w 100 & 200

Let 'm' is the order of a leaf node of B+ Tree

- Maximum number of keys a leaf node can have is = m
- Minimum number of keys a leaf node can have is = $\lceil m/2 \rceil$

{Except when root node is leaf node} in that case Min. no. of keys can be = 1

- There will be exactly one block pointer in each leaf node, it will point to the next leaf node of B+ tree. (because of this pointer range query is more efficient in B+ tree)

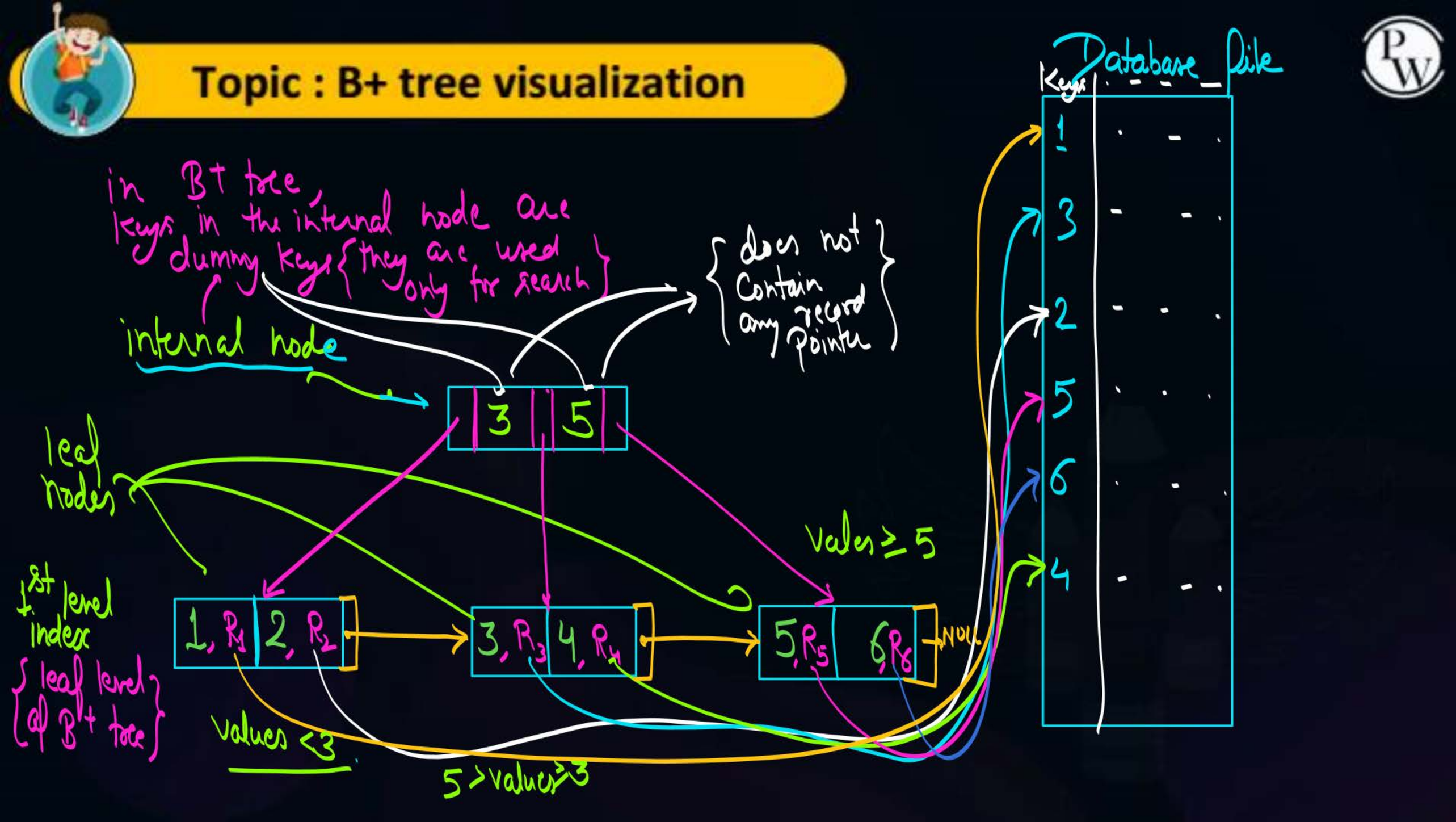
Eg: Access all records with keys from 26 to 75



Topic : NOTE

- If only one order is given in the question, then we will use the same order for both leaf node as well as for internal node.

Topic : B+ tree visualization





Topic : Insertion in B+ tree

- Insertion will be performed at leaf node, therefore search for appropriate leaf node

1. If leaf node is not full then insert the key into the leaf node in ascending order.
2. If leaf node is full,
   a) Insert the key in the ascending order.
   b) Split the node into two parts, and copy the median position key into the parent node, Because of this if there is overflow in the parent node then split the parent node as well.

because key must be present at leaf node as well.

while splitting the parent node median position key will be propagated to parent node of that not (not copied)
↳ They are used only for search

#Q. Let,

Order of internal node of B+ tree = 3, and

Order of leaf node of B+ tree = 2

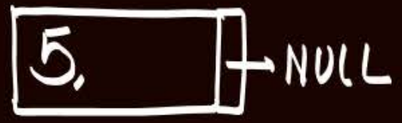
Insert 5, 15, 25, 35, 45 into the B+ tree in the same order

Max. No. of child Pointer = 3
Max No. of keys = 2
Min no. of Child Pointer : $\lceil \frac{3}{2} \rceil = 2$
Min no. of keys = 1

∴ Maximum no. of keys in leaf node = 2
& Minimum no. of keys in leaf node : $\lceil \frac{2}{2} \rceil = 1$

5, 15, 25, 35, 45

insert (5) ⇒



{ Root as well as leaf node }

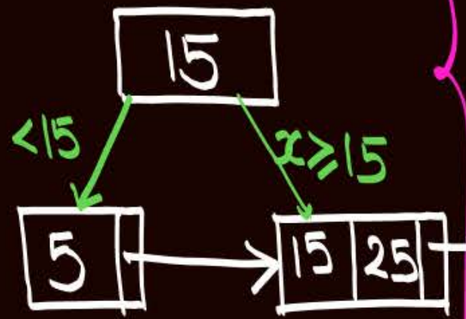
insert (15) ⇒



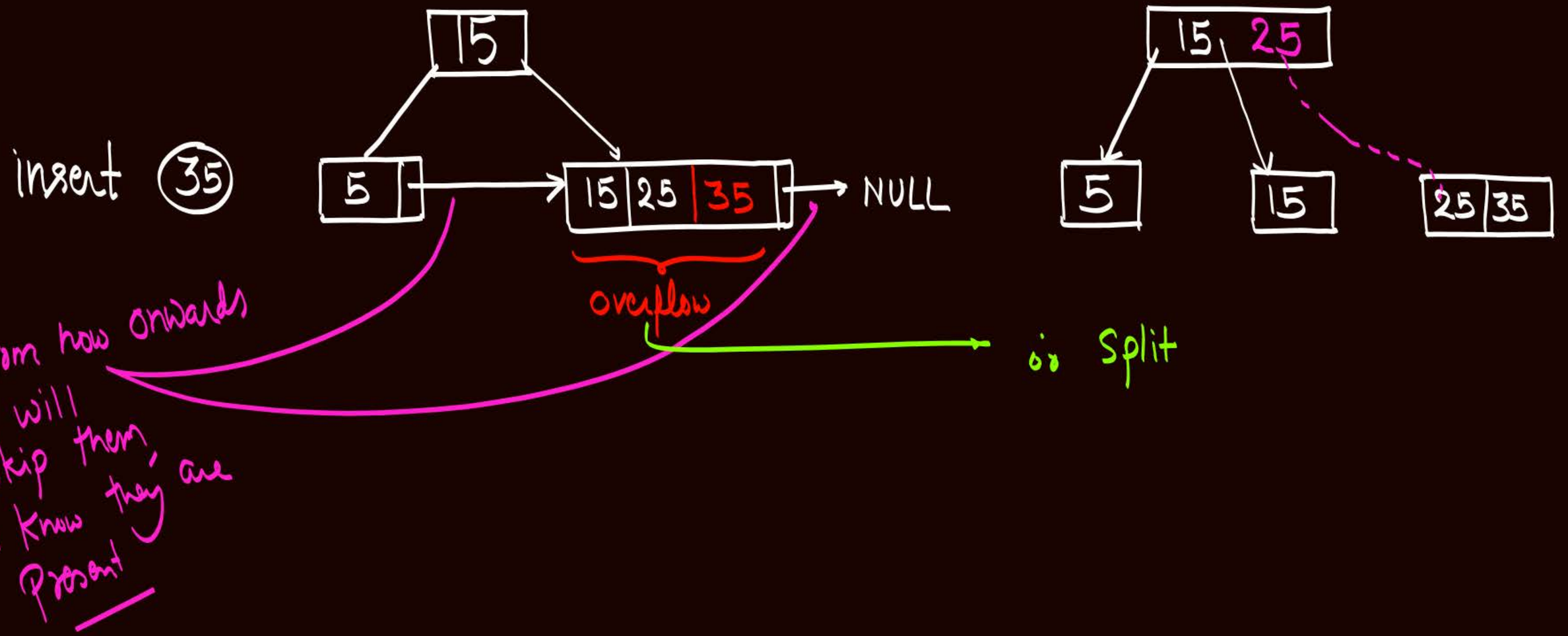
insert (25) ⇒

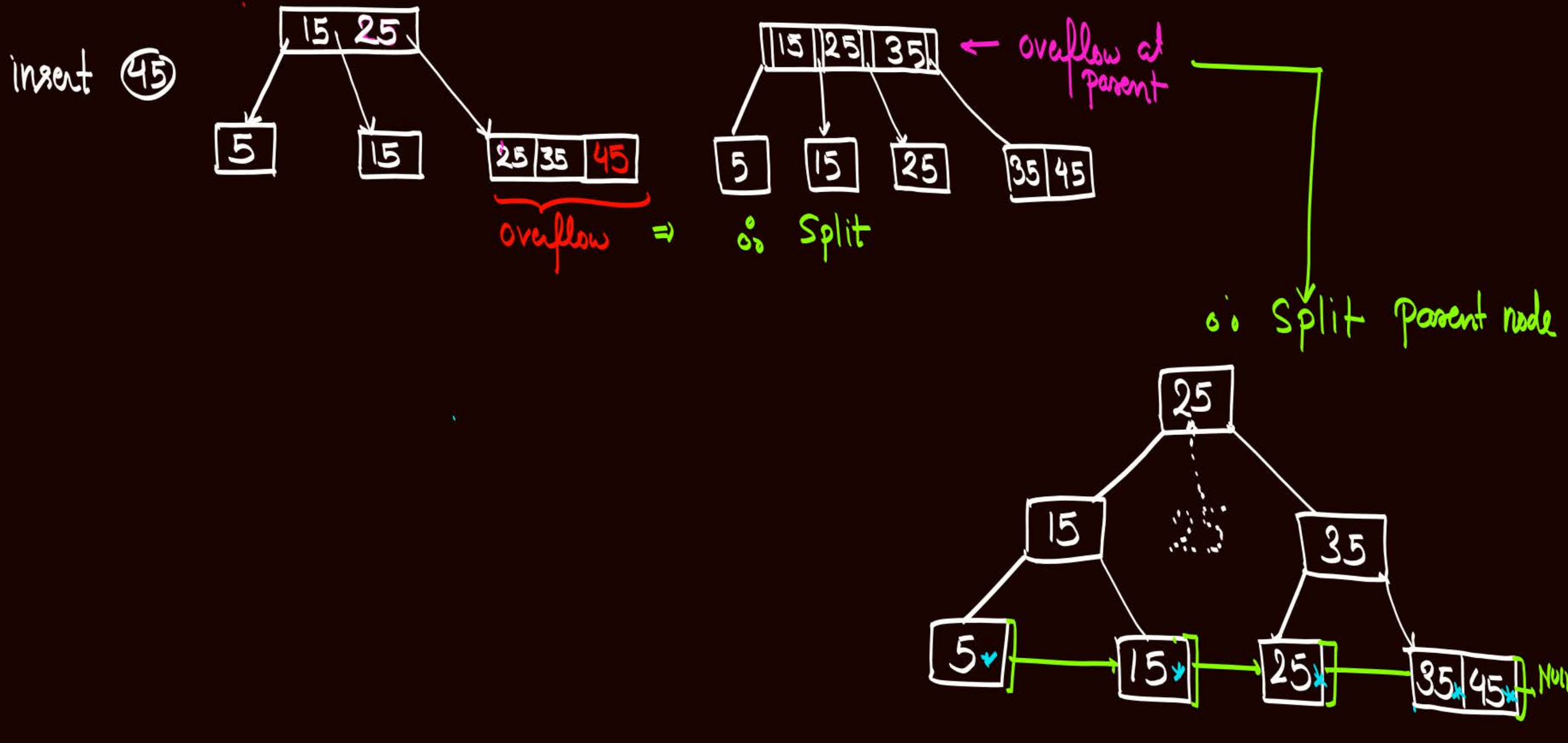


overflow ⟹ ∴ Split



{ We have assumed that element with equality condn will go into Right Subtree


insert (35)
15
5
15 25 35
NULL
overflow
from now onwards
we will
skip them
we know they are
present
or Split
15 25
5
15
25 35


insert (45)
15 25
5
15
25 35 45
overflow
∴ Split
15 25 35
overflow at parent
5
15
25
35 45
∴ Split Parent node
25
15
35
5
15
25
35 45



Topic : Deletion in B+ tree

While deleting a key from B+ tree there are two different cases.

Case 1 : When key to be deleted is present only at a leaf node.

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (Keys in the internal node are just dummy keys)



Topic : Deletion in B+ tree

Case 1 : When key to be deleted is present only at a leaf node.

1. If leaf node contains more than the minimum number of keys required, then simply delete the key from leaf node
2. If leaf node contains exact minimum number of keys required, then delete the key and borrow from the immediate siblings if they can help (left then right order).
   - When we borrow from the sibling then key at the internal node may require updation
     - inorder successor (or) Predecessor
3. If siblings can not help then merge with one of its sibling. Because of this merging if parent node becomes deficient then parent node will borrow from its immediate siblings (if possible) or merge with one of its siblings it can go up to root node.
   - if siblings can not help



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

1. If leaf node contains more than the minimum number of keys required, then simply delete the key from leaf node as well as from internal node. Because of deletion from internal node an empty space will get created, fill that empty space of internal node with in-order successor (or predecessor)

when

$k_1 \leq$ keys in the Block $< k_2$

$k_1 <$ keys in block $\leq k_2$



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

2. If leaf node contains exact minimum number of keys required and any of the immediate sibling can help, then simply delete the key from leaf node as well as from internal node, borrow the key from the sibling and fill that empty space of internal node with in-order successor (or predecessor)



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

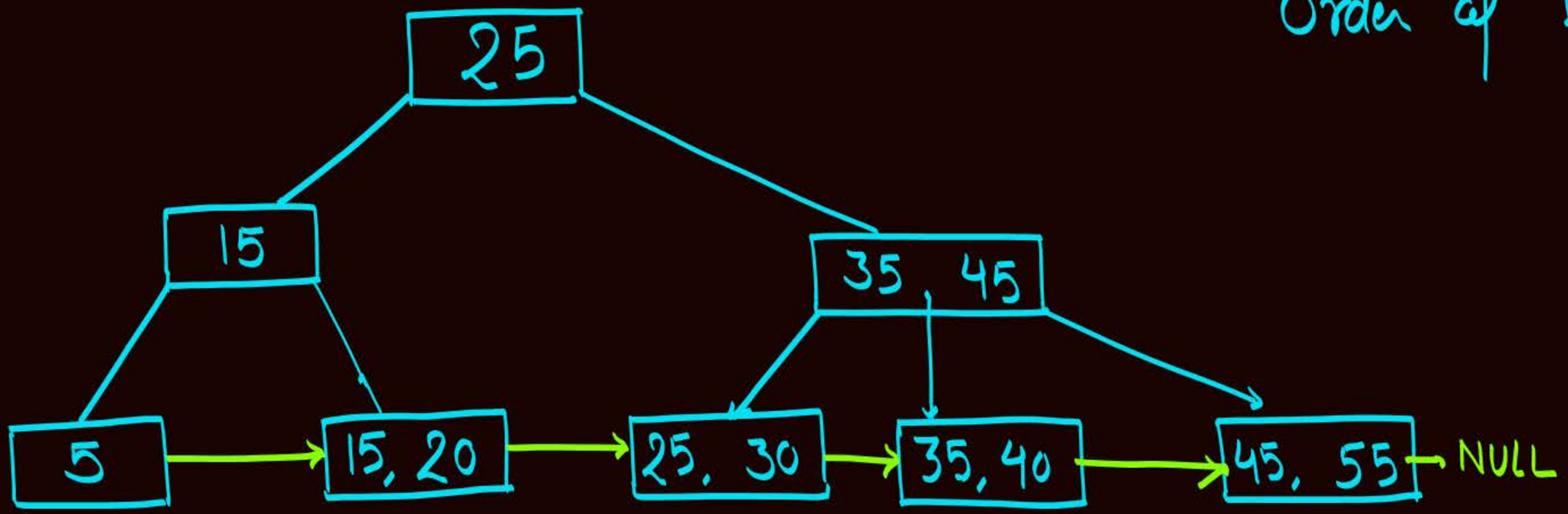
3. If leaf node contains exact minimum number of keys required and none of the immediate sibling can help, then simply delete the key from leaf node as well as from internal node and merge the leaf node with one of its sibling, it may create deficiency in the parent node therefore repeat the process for parent node as usual.

   If empty space created in internal node is not handled while merging the parent node then fill it with in-order predecessor (or successor).

Successor (or predecessor)

Q:- Consider the following B+ tree

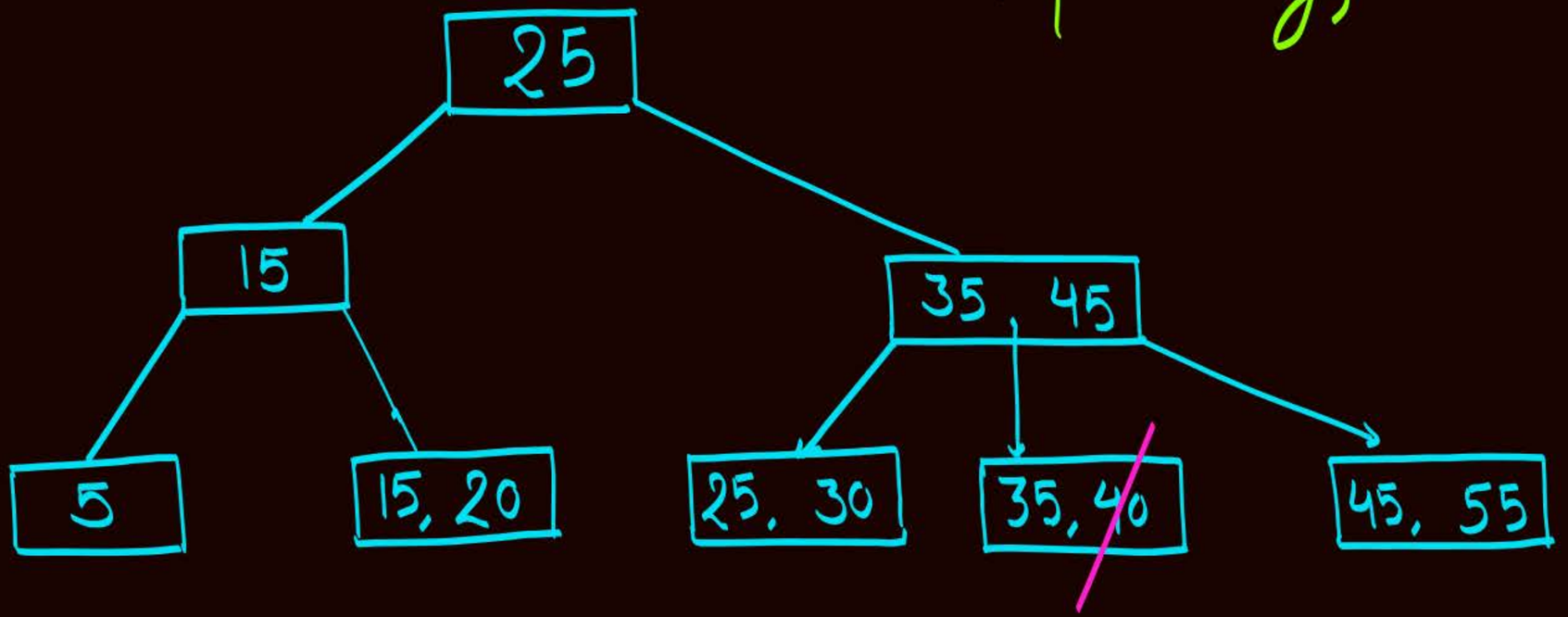
order of internal node = 3
order of leaf node = 2



delete the following elements from the above B+ tree

40, 5, 45, 35, 25, 55

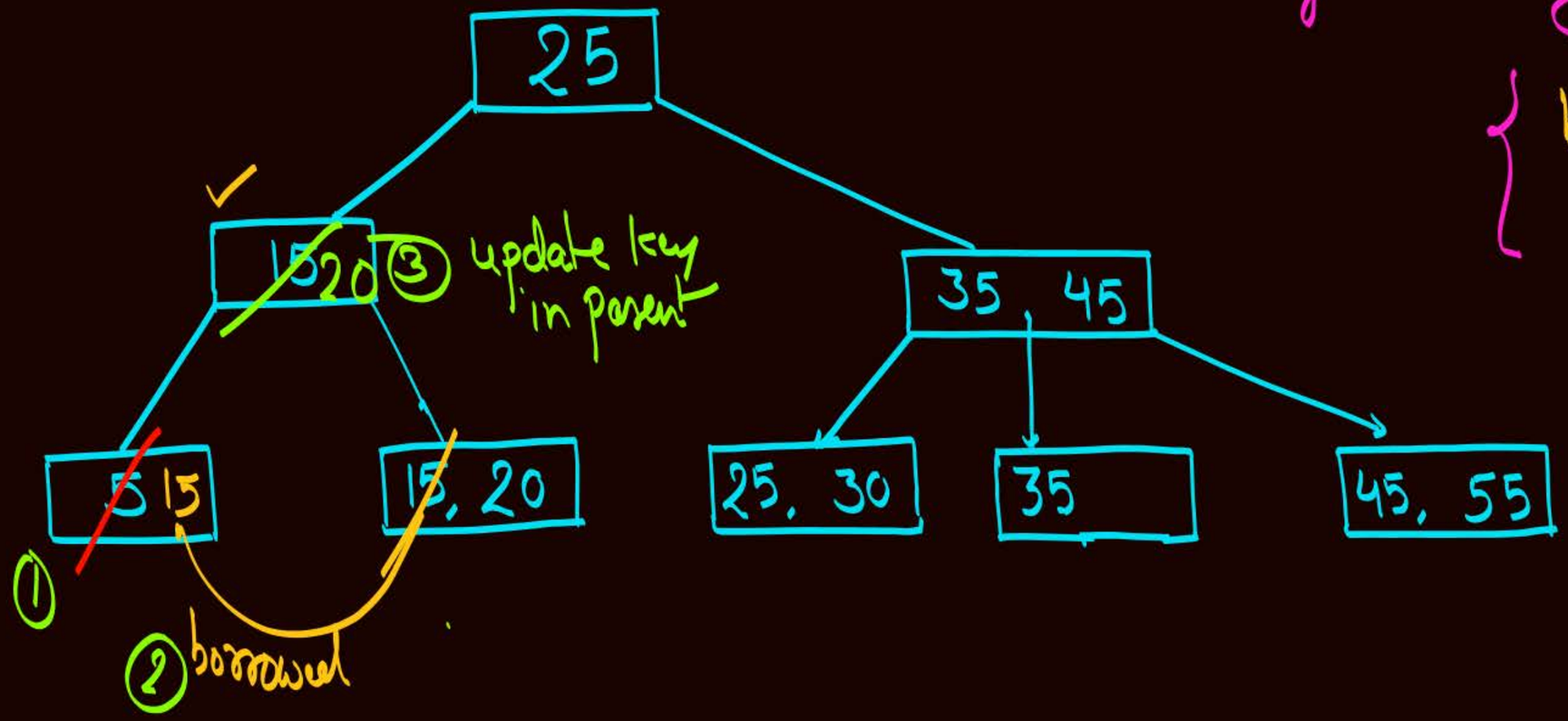
delete (40) → Present only at leaf node & node contains more than minimum required keys } ∴ Simply delete



delete (5)

only at leaf node,
but, node contain Exact Min. no. of keys required
↳ good thing is sibling can help.

{ borrow from right sibling
↳ and update key in parent node

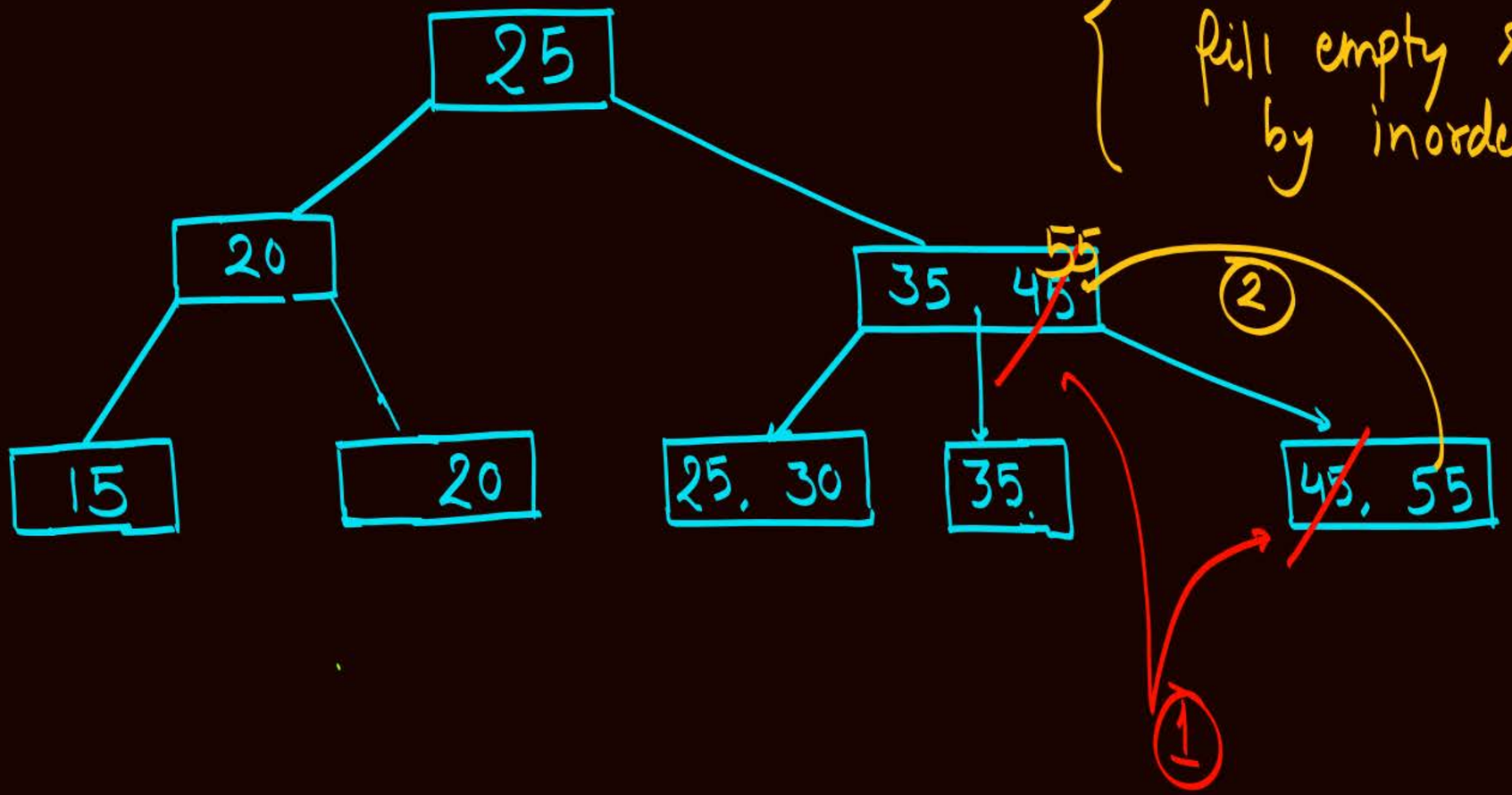


delete (45)

→ node is present at leaf as well as internal node

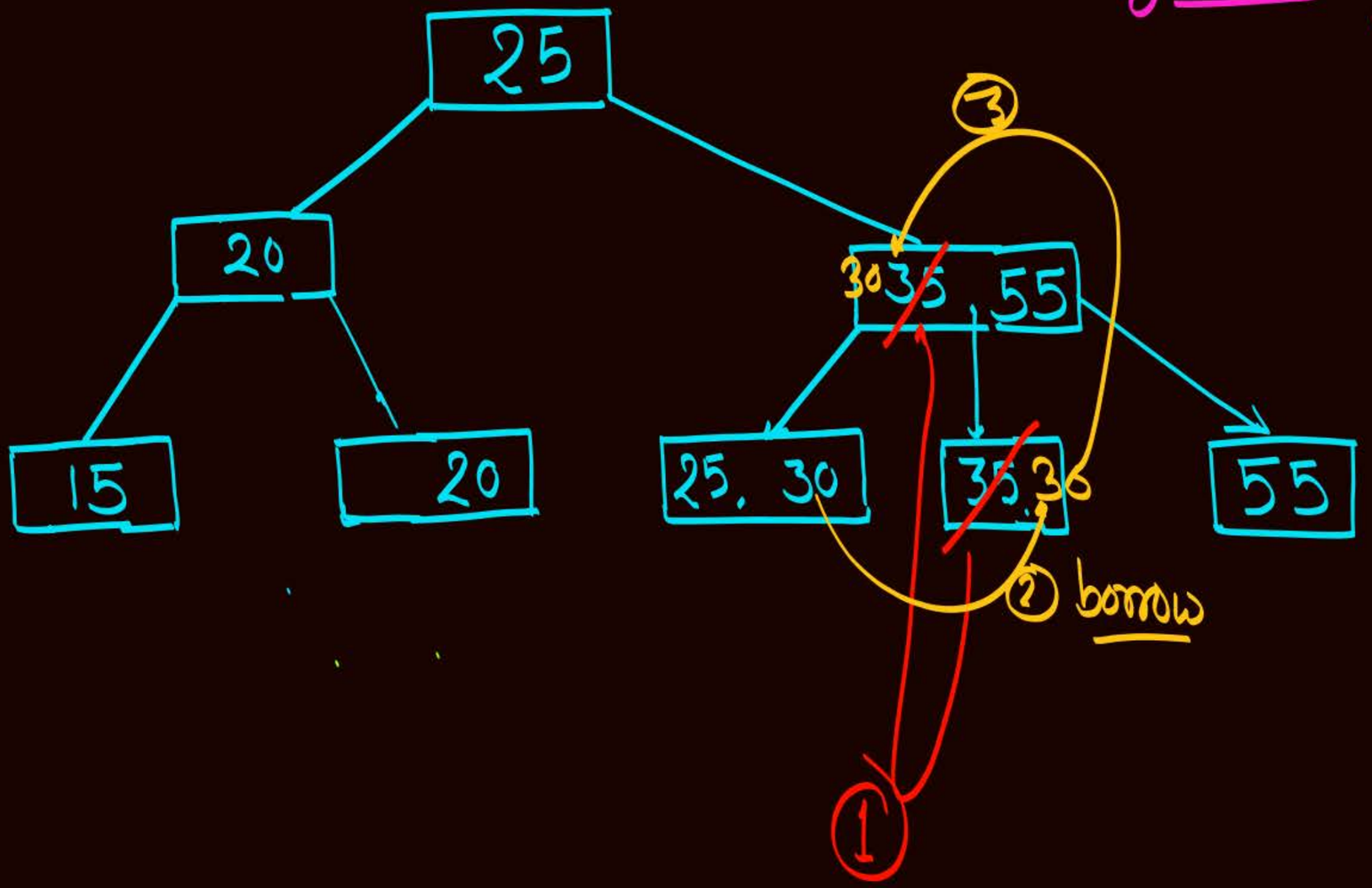
leaf node contains 4 more than minimum required keys

{ i. Delete from both places &
fill empty space in internal node
by inorder successor



delete (35) :- Key is present at both places,
leaf node contains exact minimum no. of keys required

good thing : left sibling can help

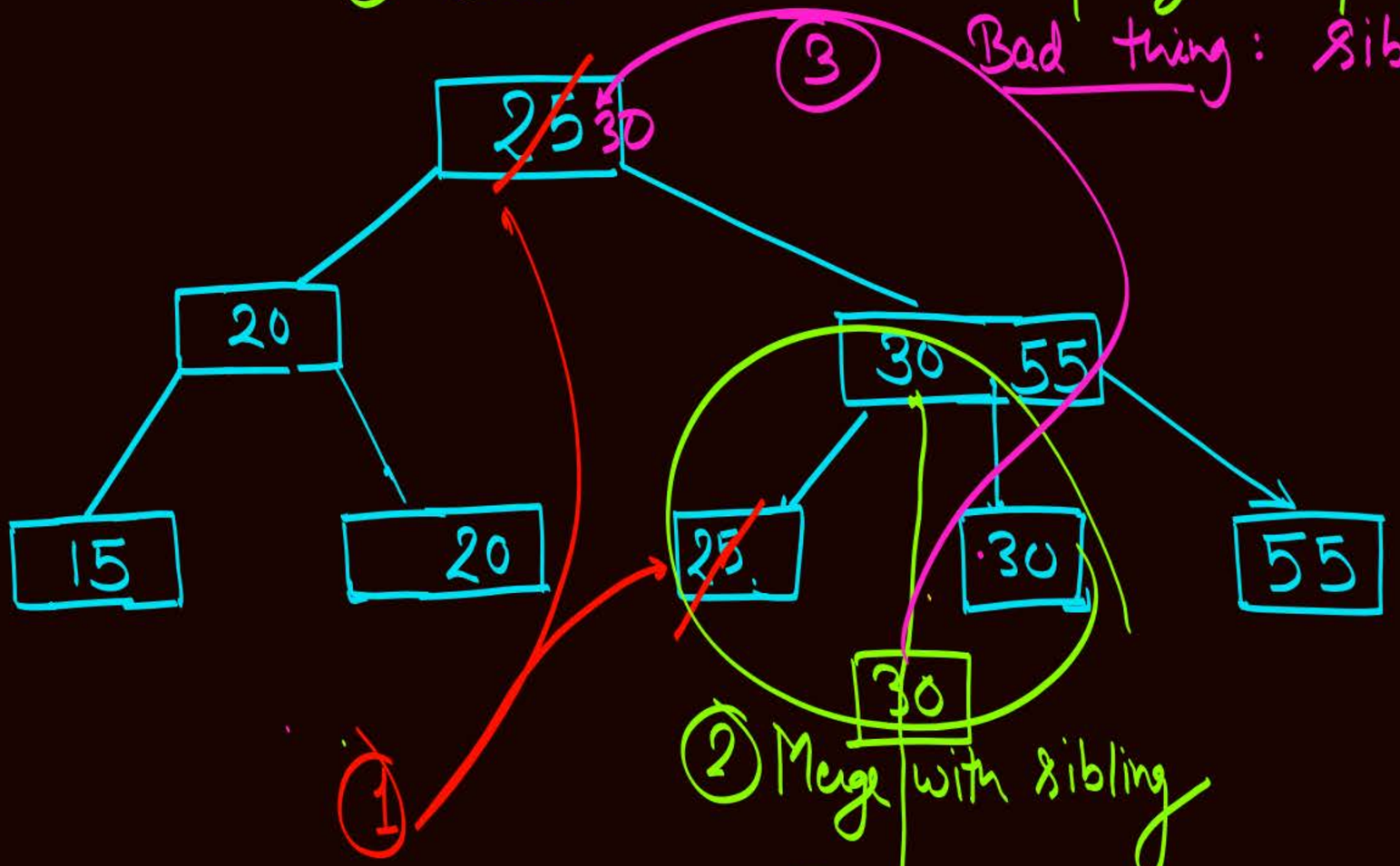


delete (25) :

① Present at both places

② Exact Exact Min no. of keys required

Bad thing : Siblings can not help



∴ ① Delete from both places

② Merge with sibling

③ Parent is deficient because of merging

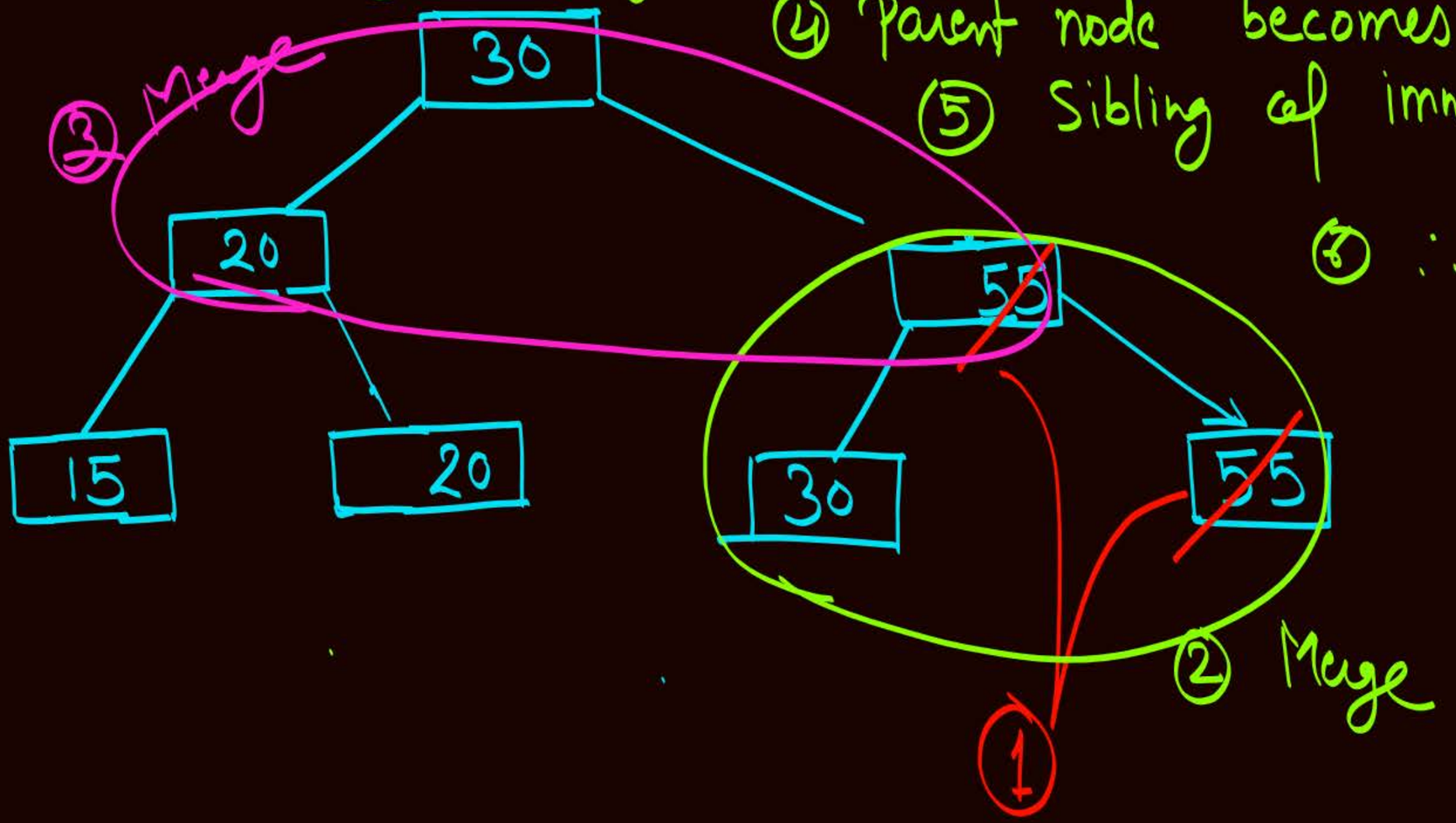
∴ Simply fill empty space by in-order successor

② Merge with sibling

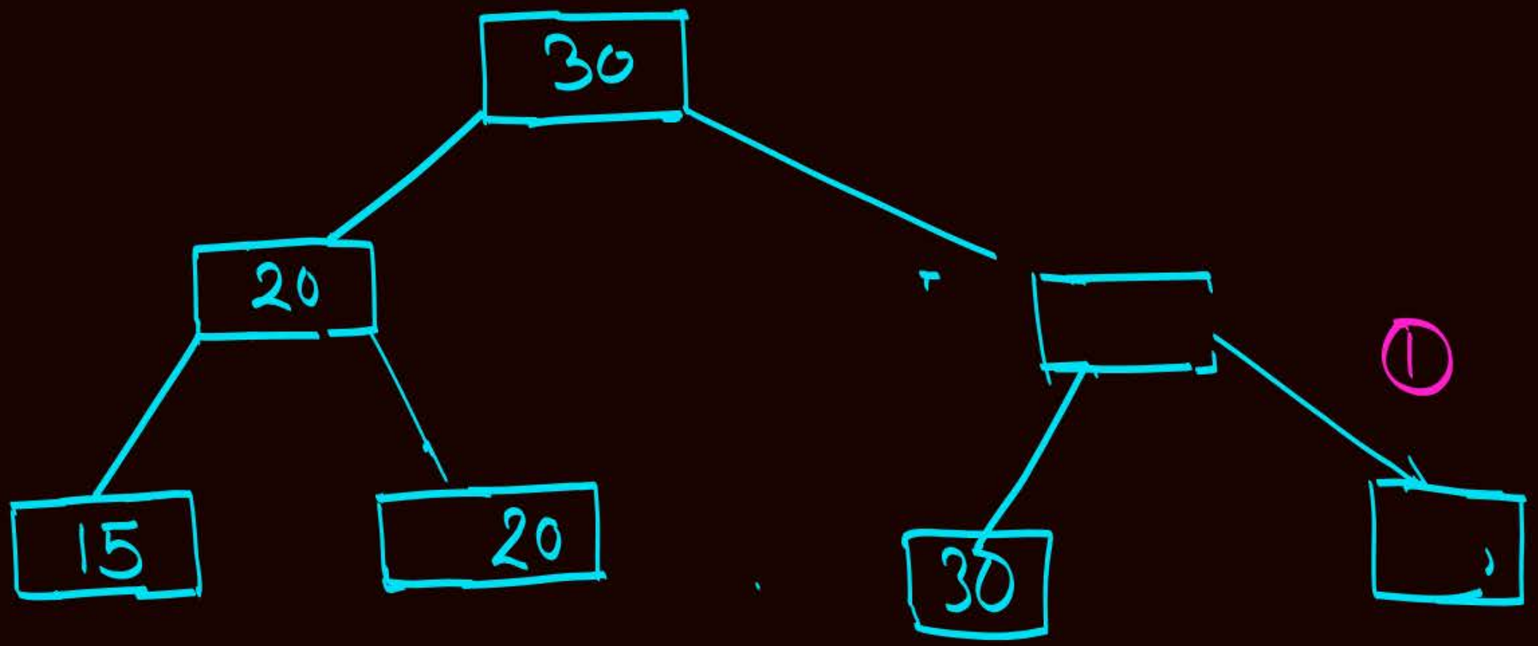
Dummy : will not appear after merging

delete (55)

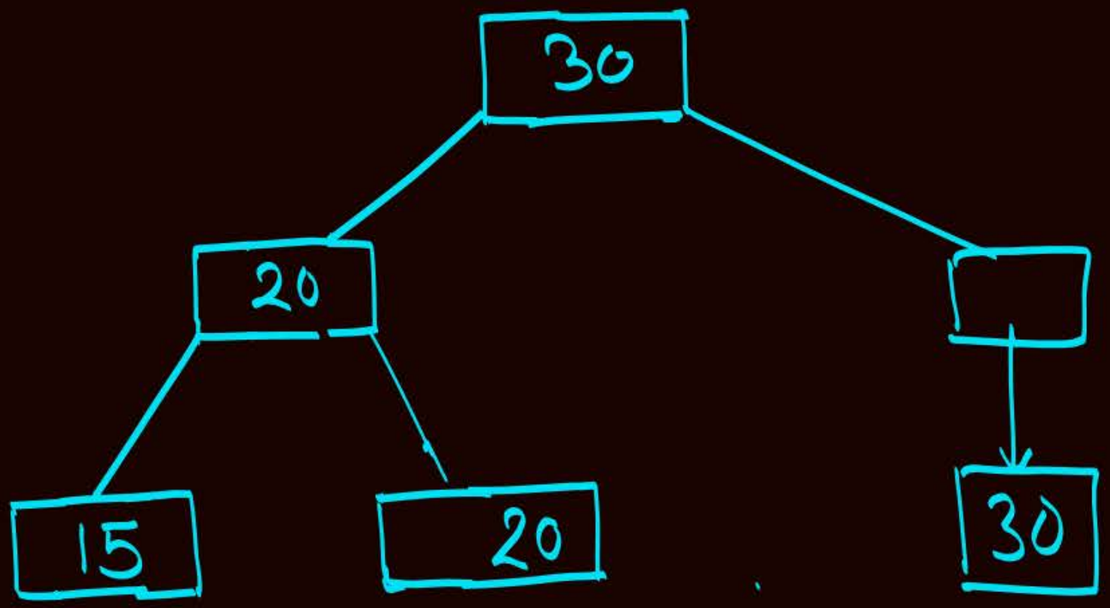
① Present at both places
② node contains exact Min no. of keys
③ Sibling can not help ∴ Merge
④ Parent node becomes deficient
⑤ Sibling of immediate Parent Can not help
⑥ ∴ Parent will merge with its sibling



delete

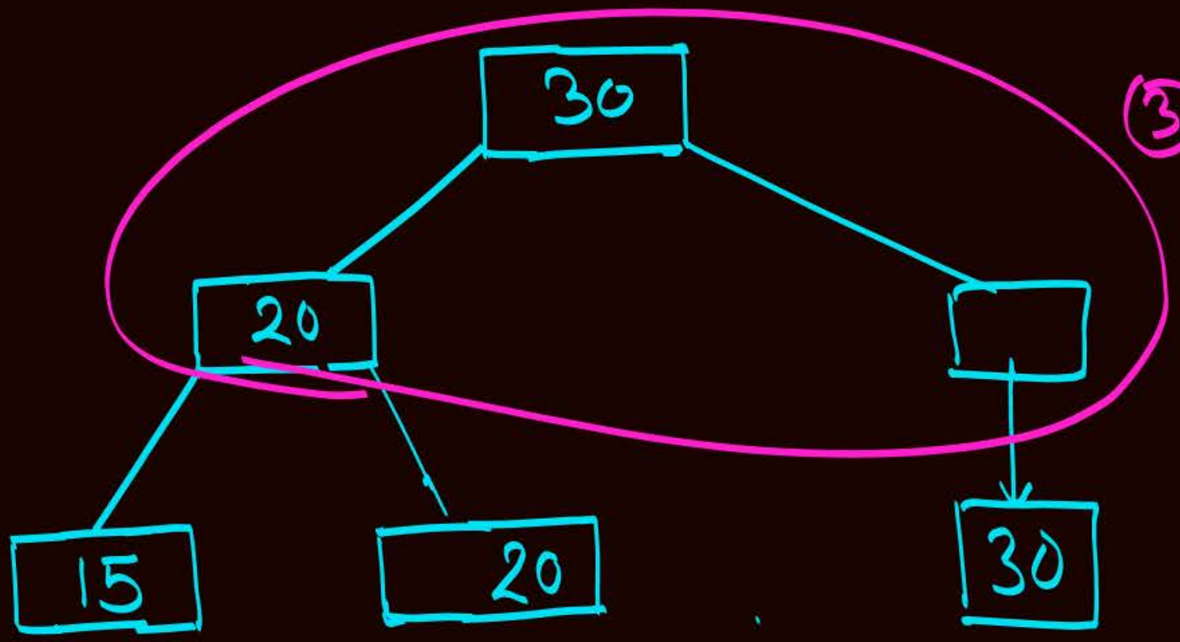


delete



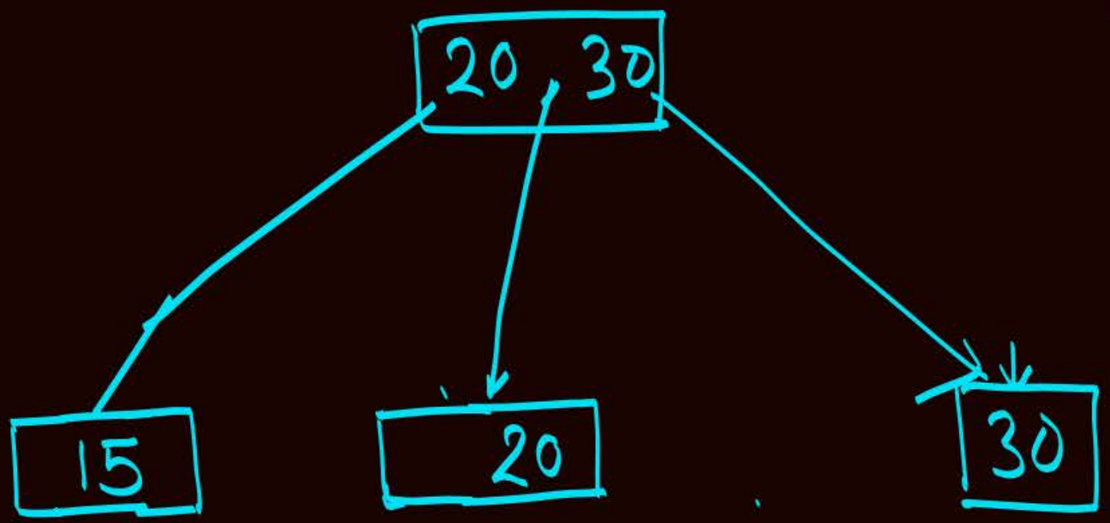
② Merged with Sibling
&
Parent is deficient

delete



③ parent merge with sibling

delete



③ parent merge with sibling



2 mins Summary

Topic B+ Tree Index

Slide

THANK - YOU